

NammaPG – ORM & JPA Concepts (Post User Entity)

1. What is an Entity?

An Entity is a Java class annotated with `@Entity` that maps to exactly one database table. Class fields map to columns and objects map to rows.

2. `@Entity` vs `@Table`

`@Entity` instructs Hibernate to create a table.

`@Table(name='roles')` defines the exact table name. It does not create a second table.

3. Role Entity – Why Separate Table

Roles define authority, not users. A role can be shared across users, hence roles must be stored separately.

4. Why Three Tables for User–Role

users → stores user data

roles → stores role definitions

user_roles → join table for many-to-many relationship

5. Lombok Annotations

`@Getter` – generates getter methods

`@Setter` – generates setter methods

`@NoArgsConstructor` – mandatory for Hibernate

`@AllArgsConstructor` – optional and often avoided

`@Builder` – preferred for safe object creation

6. Why No-Args Constructor Is Required

Hibernate creates empty objects using reflection and then injects values field by field.

7. Builder vs All-Args Constructor

Builder is readable, order-independent, and scalable.

All-args constructors become error-prone as entities grow.

8. Dependencies, Packages, Imports

pom.xml is a configuration file.

Dependencies are libraries declared in pom.xml.

Libraries contain packages.

Packages contain classes.

9. jakarta.persistence vs Lombok

jakarta.persistence → ORM annotations (JPA specification)

Lombok → Java boilerplate reduction

Hibernate → ORM implementation

Status

✓ MySQL connected

✓ User entity created

✓ ORM concepts locked

Next: Implement User–Role mapping (@ManyToMany).